Proyecto de comprobación de C: *runlog*

Reto 358,5 días · Fase 0 · Capítulos 1-6 de la Guía de Beej · Dificultad media-alta

1. El caso real

Eres speedrunner. Practicas **5 misiones** de un juego y para cada una guardas hasta **10 intentos**, con el tiempo en segundos. Un intento no jugado (o abandonado) se marca con `-1`. Quieres un programa de consola que te diga en qué misión estás peor, cuál es tu mediana, si estás mejorando y cómo se reparten tus tiempos.

Con los capítulos 1-6 todavía **no tienes** cadenas, structs ni archivos, así que todo el diseño se hace con **arrays, punteros y funciones**. Los datos viven en memoria:

```
int tiempos[MISIONES][INTENTOS];   /* segundos; -1 = intento no valido */
```

Cada misión se identifica por su número (0-4). Esa restricción es parte del ejercicio: en el capítulo 7 descubrirás lo que te falta.

2. Requisitos obligatorios

1. **Sin variables globales** modificables. Todo se pasa a las funciones como argumento. Sí puedes usar `#define` para constantes (`MISIONES`, `INTENTOS`).
2. Una función de estadísticas que **devuelva varios resultados mediante punteros**, con un prototipo parecido a: `int estadisticas(const int *fila, int n, int *min, int *max, double *media)`, donde el retorno es el nº de intentos válidos. Si no hay ninguno, no debe dividir entre cero ni escribir valores basura.
3. Una función `intercambiar(int *a, int *b)` y un **ordenamiento propio** (burbuja o inserción) que la use. Se aplica a una **copia** de la fila para calcular la mediana sin destruir los datos originales (la copia la haces tú con un bucle).
4. Pasar el array 2D a funciones con su dimensión correcta (`int t[][INTENTOS]`) y recorrerlo con bucles anidados.
5. **Ranking de misiones** por media de tiempo, de peor a mejor, usando un array auxiliar de índices que ordenas (no se reordenan los datos).
6. **Racha de mejora**: cuántos intentos consecutivos válidos de una misión han sido más rápidos que el anterior (ignorando los `-1`). Es un problema de estado dentro de un bucle: diseña bien el caso del `-1` en medio.
7. **Histograma ASCII** de una misión: agrupa los tiempos en tramos (por ejemplo, de 30 en 30 s) y dibuja una barra de # por tramo.
8. **Generación de datos de prueba** con números pseudoaleatorios (parte de ellos inválidos) y también **carga manual** de un tiempo concreto desde teclado.
9. Un menú con `do-while` y `switch`: generar datos, registrar un tiempo, estadísticas de una misión, ranking, histograma, salir.
10. **Validación de entrada**: nunca puede romperse con letras, números fuera de rango o un índice de misión inexistente. Escribe una función `leer_entero(int min, int max)` que repita hasta recibir un valor correcto.
11. Compilar con `gcc -Wall -Wextra -std=c2x -pedantic` sin warnings, y funciones pequeñas con prototipos arriba del archivo.

12 Devolver `EXIT_SUCCESS` al salir con normalidad.

3. Te fuerzan a consultar documentación

Nada de esto está en los capítulos 1-6 (o solo se menciona de pasada). Busca cada punto y anótalo en tu cuaderno:

- `scanf`: qué devuelve, por qué hay que comprobar ese valor y qué ocurre con lo que el usuario escribió y `scanf` no consumió. Cómo vaciar el buffer de entrada (pista: `getchar`).
- `rand`, `srand` y `time(NULL)` (el capítulo 3 lo menciona en una nota al margen). Cómo obtener un número en un rango [a, b] y qué es el sesgo del módulo.
- `INT_MAX` e `INT_MIN` en `<limits.h>`: para qué sirve inicializar un mínimo con un valor enorme.
- División entera frente a división real, y el **cast** `(double)` (capítulo 15, adelantado).
- El truco `sizeof array / sizeof array[0]` y por qué **no funciona** dentro de la función que recibe el array (el capítulo 6 te lo explica).
- El calificador `const` en parámetros puntero (capítulo 16, adelantado).
- `printf` con anchos y precisión: `%4d`, `%-8s`, `%6.2f`, `%%`, y `%zu` para `size_t`.
- Operaciones sobre arrays fuera de límites: qué es comportamiento indefinido y cómo detectarlo con `-fsanitize=address,undefined` al compilar.

Fuentes: `man 3 scanf`, cppreference.com y beej.us/guide/bgclr.

4. Casos de prueba

Pruébalos uno a uno y anota qué hace tu programa:

- Misión con **todos** los intentos en `-1` (ningún dato válido).
- Misión con **un solo** intento válido (media, mediana y mínimo coinciden).
- Misión con todos los tiempos iguales (la racha de mejora debe ser 0).
- Cantidad **par** de intentos válidos (la mediana es la media de los dos centrales) e **impar**.
- Intentos válidos separados por `-1` en medio (la racha no debe romperse por un hueco... o sí: decide y documenta).
- Usuario escribe letras donde se pide un número, o pulsa Enter sin escribir nada.
- Misión 7 cuando solo hay 5; intento 11 cuando solo hay 10; tiempo 0 o negativo distinto de `-1`.
- Ejecutar el menú 50 veces seguidas sin que el programa se degrade.

5. Documentación que debes entregar

Un `README.md` con:

1 Qué hace y cómo se usa, con una salida de ejemplo pegada.

2 **Decisiones de diseño** (mini-ADR: decisión, alternativa descartada, motivo). Por ejemplo: por qué `-1` como centinela, cómo defines la racha, por qué ordenas una copia.

3 **Cuaderno de errores**: cada bug serio, cómo lo encontraste y cómo lo arreglaste.

4 **Limitaciones conocidas** y qué capítulo las resolvería: nombres de misión (cadenas, cap. 7), agrupar datos (structs, cap. 8), guardar en disco (archivos, cap. 9), tamaño variable (`malloc`, cap. 12).

6. Rúbrica de autoevaluación

Nivel	Qué significa
Aprobado	Compila sin warnings y calcula bien con el caso "feliz".
Notable	Supera todos los casos de prueba sin dividir entre cero ni acceder fuera de límites; el menú no se rompe.
Sobresaliente	Usa `-fsanitize` sin errores, funciones con una sola responsabilidad y README completo con cuaderno de errores.

7. Hitos sugeridos

1. Declarar el array 2D, rellenarlo a mano y mostrarlo con `printf` alineado.
2. Función `estadisticas` (min, max, media) para una fila y probarla con los casos raros.
3. Copia, ordenamiento y mediana.
4. Ranking con el array de índices y racha de mejora.
5. Histograma y generación aleatoria.
6. Menú con validación de entrada.
7. Casos de prueba, sanitizers y README.

Cuando termines, comparte el código para revisarlo juntos. Pasará a tu bitácora como hito de la Fase 0.